

```
////////////////////////////////////  
/////Practice part////////////////////////////////////  
////////////////////////////////////
```

Symbols:

#: command
//: description

```
//Make a new directory  
# mkdir practice  
//Change directory  
# cd practice
```

```
# mkdir Downloads  
# mkdir Pictures  
# mkdir Movies  
# mkdir Desktop  
# mkdir Documents  
# mkdir Music  
# mkdir Videos  
# mkdir Templates
```

```
//Check  
//Extended ls command  
# ls -la
```

```
# cd Music
```

```
//Check the location  
# pwd
```

```
# cd ../Videos
```

```
//Check the location  
# pwd
```

```
//Go up  
# cd ..
```

```
//Check the location  
# pwd
```

```
//List directories with start "D" character
# ls D*
# ls D* -l -d
```

```
//List files and directories with start
// "D", include "n" and end with "s"
# echo D*n*s
```

```
//List files and directories
//Start with "D" and 9 character long
// 9x ? character
# echo D?????????
```

```
//Copy /etc/hosts file to my home folder
# cp /etc/hosts hosts
```

```
//Check
# ls -la
```

```
//Create a backup file
# cp hosts hosts_backup
```

```
//Delete the original "hosts" file
# rm hosts
```

```
//Compress
```

```
//Move "hosts_backup" file to "Movies"
//directory
# mv hosts_backup Movies
```

```
////////////////////////////////////
//Today's new commands////////////////////////////////
////////////////////////////////////
```

//The 'tar' command is used to merge multiple files into a single file. By default, it does not compress the data.

```
//The -c option tells the tar command to create a tar file.
//The -v option stands for "verbose", which instructs the tar command to demonstrate what it is
doing.
//The -f option is used to specify the name of the tar file.
//The -t option is used to list contents.
//Create archive file
# tar -cvf movies.tar Movies/
```

```
//Check
# ls -la
```

```
//Delete the directory include files
# rm -r Movies
```

```
//Check
# ls -la
```

```
//For unzuip/uncompress "movies.tar" file
# tar -xvf movies.tar
```

```
echo "hello" > sample.txt
echo "hello again" >> sample.txt
cat sample.txt
```

```
cat /etc/passwd > sample.txt
cat /etc/passwd >> sample.txt
```

```
//cut command: extract all of the usernames from a database called /etc/passwd
# cut -d: -f1 /etc/passwd
```

```
//The Pipe is a command in Linux that lets you use two or more commands such that output of
one command serves as input to the next. In short, the output of each process directly as input
to the next one like a pipeline. The symbol '|' denotes a pipe.
#
```

```
//The output in the previous example was unordered and scrolled off the screen. In the next step
you are going to take the output of the cut command and send it into the 'sort' command to
provide some order to the output
//SORT command is used to sort a file, arranging the records in a particular order
# cut -d: -f1 /etc/passwd | sort
```

//Now the output is sorted, but it still scrolls off the screen. Send the output of the sort command to the more command to solve this problem:

```
# cut -d: -f1 /etc/passwd | sort | more
```

//You can use the 'head' command to display the top part of a file.

```
# head /etc/passwd
```

//Use the 'tail' command to display the last ten lines.

```
# tail /etc/passwd
```

//Use the head command to display the first two lines

```
# head -2 /etc/passwd
```

///GREP command////////////////////

///Module 10////////////////////

In this task, you will use the 'grep' family of commands with regular expressions in order to search for a specific string of characters in a stream of data (for example, a text file).

The 'grep' command uses basic regular expressions, special characters like wildcards that match patterns in data. The 'grep' command returns the entire line containing the pattern that matches.

The '-E' option to the 'grep' command can be used to perform searches with extended regular expressions, essentially more powerful regular expressions. Another way to use extended regular expressions is to use the egrep command.

The 'fgrep' command is used to match literal characters, ignoring the special meaning of regular expression characters.

```
# cd /etc
# grep sshd passwd
# grep root passwd
```

//^ character can be used to match a pattern at the beginning of a line

```
# grep '^root' passwd
```

//Match the pattern sync anywhere on a line:

```
# grep 'sync' passwd
```

//\$ symbol to match the pattern sync at the end of a line

```
# grep 'sync$' passwd
```

```
//Use the period character . to match any single character
```

```
//For example, execute the following command to match any character  
followed by a 'y'
```

```
# grep '.y' passwd
```

```
//The pipe character, |, or "alternation operator", acts as an "or" operator. For example, execute  
the following to attempt to match either sshd, root or operator.
```

```
//Use the -E switch to allow grep to operate in extended mode in order to recognize the  
alternation operator
```

```
# grep -E 'sshd|root|operator' passwd
```

```
//Use another extended regular expression, this time with egrep with alternation in //a group to  
match a pattern. The strings nob and non will match
```

```
# egrep 'no(b|n)' passwd
```

```
//The [ ] characters can also be used to match a single character. However, unlike the period  
character ., the [ ] characters are used to specify exactly what character you want to match. For  
example, if you want to match a numeric character, you can specify [0-9]
```

```
# head passwd | grep '[0-9]'
```

```
//Suppose you want to search for a pattern containing a sequence of three digits. You can use {  
} characters with a number to express that you want to repeat a pattern a specific number of  
times; for example: {3}
```

```
# grep -E '[0-9]{3}' passwd
```